Day1.

파이썬 개발환경 구축 및 코드 작성방법

목 차

1. 파이썬 소개 3

2. 컴퓨터 프로그램 언어의 구분 6

3. 파이썬 개발환경 설치 9

4. Anaconda 설치 및 가상환경 만들어 사용하기 17

5. Jupyter Notebook 20

6. Git 에 개인 프로젝트 관리하기 25

1. 파이썬 소개

파이썬(Python)은 1991 년 귀도 반 로썸(Guido van Rossum)이 개발한 **고급 프로그래밍 언어**로, **읽기 쉽고 간결한 문법**을 특징으로 하는 대표적인 **인터프리터 언어**입니다.

- **인터프리터 언어**

  Interpreter 는 해석기라는 뜻으로, 소스 코드를 컴파일하지 않고 코드를 한 줄씩 읽어서 바로 실행하는 프로그램 방식의 언어입니다.

- **귀도 반 로썸**

  파이썬을 창시하여 현대 IT 및 AI 시대를 이끈 핵심 인물

✓ 1974 년도에 University of Amsterdam 에 진학하여 Computer Science 를 전공

✓ 1982 년도부터 Mathematical Center (지금은 CWI research institute)에서 일함

✓ ABC 언어 개발에 참여

✓ 파이썬 언어 개발을 시작
  ▶ 유닉스 기반의 ABC 언어는 많은 사용자를 확보하는 데에 실패함

✓ 1994 년, 함수형 프로그래밍, 문자열 처리 기능 등을 추가한 Version 1.0 공개

✓ 컴퓨터에 대한 전문적인 지식이 부족한 과학자, 조교 등의 사용자들이 편하게 쓸 수 있는 프로그래밍 언어 개발

✓ 문법이 간결하고 사용하기 편리하며 높은 확장성을 갖춘 언어로 목표 설정

✓ 1989 년 크리스마스에 새로운 프로그래밍 언어의 개발을 시작

✓ 영국의 코메디 그룹인 몬티 파이썬 (Monty Python)에서 이름 차용

✓ ABC 언어의 특징을 계승하여 1991 년 2 월에 Version 0.9.0 공개

✓ CWI 를 설득해서 오픈 소스 형태로 공개

✓ 대표적인 철학:

"Simple is better than complex." (단순함이 복잡함보다 낫다)

• 파이썬(Python)의 특징

파이썬은 **간결한 문법과 강력한 라이브러리**를 기반으로 다양한 분야에서 사용되는 대표적인 프로그래밍 언어입니다.

1. 쉬운 문법과 높은 가독성

- 영어와 유사한 문장 구조
- 들여쓰기(Indentation)로 코드 블록 구분
- 코드가 간결하고 이해하기 쉬움

```
if score >= 60:
    print("합격")
```

2. 인터프리터 언어

- 코드를 **한 줄씩 해석하여 실행**
- 컴파일 과정 없이 빠른 실행 가능
- 테스트 및 디버깅이 용이

3. 객체지향 + 함수형 지원

- 객체지향 프로그래밍(OOP) 지원
- 함수형 프로그래밍 개념도 포함
- 다양한 프로그래밍 스타일 가능

4. 풍부한 표준 라이브러리

- 파일 처리, 네트워크, 데이터 처리 등 기본 제공
- "배터리 포함(Batteries Included)" 철학

5. 다양한 외부 라이브러리

- 데이터 분석: NumPy, Pandas
- AI/ML: TensorFlow, PyTorch
- 웹 개발: Django, Flask

6. 플랫폼 독립성

- Windows, Linux, macOS 모두 실행 가능
- 동일 코드로 다양한 환경에서 실행

7. 동적 타이핑 (Dynamic Typing)

- 변수 선언 시 자료형 지정 불필요
- 실행 시 타입 결정

```
x = 10
x = "Hello"
```

8. 자동 메모리 관리

- 가비지 컬렉션(GC) 지원
- 개발자가 메모리 직접 관리할 필요 없음

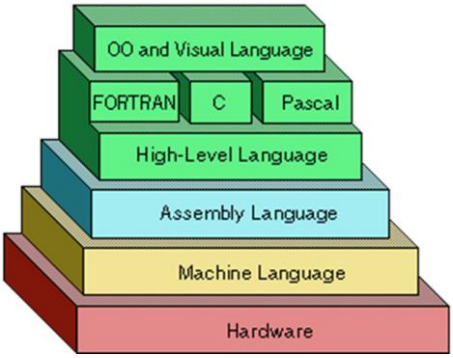
9. 확장성과 통합성

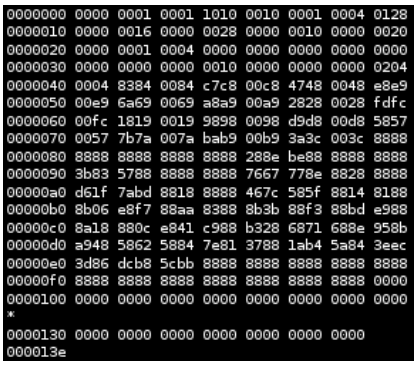
- C/C++, Java 등과 연동 가능
- 다양한 시스템과 쉽게 통합

10. 오픈소스 + 강력한 커뮤니티

- 무료 사용 가능
- 전 세계 개발자 커뮤니티 존재

2. 컴퓨터 프로그램 언어의 구분





컴퓨터 프로그램 언어는 **수준(Level), 실행 방식, 목적**에 따라 구분할 수 있습니다.

1. 수준(Level)에 따른 구분

① 저급 언어 (Low-Level Language)

컴퓨터(기계) 중심 언어

- **특징**
  - 컴퓨터가 직접 이해 가능
  - 실행 속도 빠름
  - 사람이 이해하기 어려움
- **종류**
  - **기계어 (Machine Language)**: 0 과 1 로 구성
  - **어셈블리어 (Assembly Language)**: 기계어를 기호화

    MOV AX, 1

    ADD AX, 2

② 고급 언어 (High-Level Language)

인간 중심 언어

- **특징**
  - 사람이 이해하기 쉬움
  - 생산성 높음
  - 컴파일/인터프리터 필요
- **예시**
  - Python, Java, C++

    print("Hello")

2. 번역 방식에 따른 구분

① 컴파일러 언어 (Compiled Language)

전체 코드를 한 번에 번역 후 실행

- **특징**
  - 실행 속도 빠름
  - 오류 발견은 컴파일 시점
- **예시**
  - C, C++
  - Pascal, Fortran

② 인터프리터 언어 (Interpreted Language)

한 줄씩 해석하며 실행

- **특징**
  - 개발 및 테스트 용이
  - 실행 속도 상대적으로 느림
- **예시**
  - Python
  - JavaScript

③ 하이브리드 방식

컴파일 + 인터프리터 혼합

- **예시**
  - Java, C#

    → Java 는 바이트코드로 컴파일 후 JVM 에서 실행

3. 목적에 따른 구분

① 시스템 프로그래밍 언어

- OS, 드라이버 개발
- 예: C, C++, Assembly

② 응용 프로그래밍 언어

- 일반 소프트웨어 개발
- 예: Java, Python, C#, JavaScript, Kotlin, Swift, Dart, TypeScript

③ 스크립트 언어

- 자동화, 간단한 작업
- 예: Python, JavaScript, PHP, Ruby

4. 패러다임에 따른 구분

✔ **절차지향 언어**

- 순차적 실행
- 예: C, Pascal, Fortran, COBOL

✔ **객체지향 언어**

- 객체 기반 설계
- 예: Java, C++, C#, Python

✔ **함수형 언어**

- 수학적 함수 기반
- 예: Haskell, Lisp, Scheme, ML

3. 파이썬 개발환경 설치

통합 개발 환경 (IDE)

- PyCharm
- Visual Studio Code
- Spyder
- Thonny
- Eclipse

웹 기반 개발 환경

- Jupyter Notebook
- Google Colab

실행 환경 / 인터프리터

- CPython
- Anaconda
- Miniconda
- PyPy

Python 설치

1. python OS 확인 최신버전 다운로드 > 설치

https://www.python.org/downloads/

> 다운파일 실행 : python-x.x.x-amd64.exe

> setup 창에서 아래쪽에 있는

- 체크함 : Add Python x.x to PATH
- Install Now 클릭함

> 설치완료 : close

> 설치확인 : 시작 -> 모든 프로그램 -> Python x.x 확인

VS Code 설치

Python 설치 완료 후, cmd(명령 프롬프트) 실행해서 아래 입력하여 설치 확인:
...> python --version
예) `Python x.x.x` 처럼 나오면 성공.

> macOS: `brew install python`
> Ubuntu: `sudo apt update && sudo apt install python3 python3-venv python3-pip -y`

2. VS Code 설치

1. 브라우저에서 `VS Code download` 검색.
2. Windows용 **User Installer** 다운로드 후 실행.
3. 설치 과정에서 기본 옵션 그대로 진행해도 됨.
4. 설치 후 VS Code 실행.

3. VS Code에 Python 확장(Extension) 설치

1. VS Code 왼쪽 사이드바에서 네모 네 개 아이콘 → Extensions (확장) 클릭
   (단축키: `Ctrl+Shift+X`)
2. 검색창에 `Python` 입력.
3. Microsoft에서 제공하는 "Python" 확장을 찾아 Install(설치) 클릭.
4. 추가로 설치하면 좋은 것:
   * Pylance (Python 언어 서버, 자동완성/타입 분석 좋아짐)
   * Jupyter (노트북 형식으로 코딩할 때)

4. 첫 파이썬 파일 만들고 실행해 보기

1. 작업용 폴더 만들기
   1. 윈도우에서 `C:₩python_workspace` 같은 폴더 하나 만들기.
   2. VS Code에서 메뉴 File → Open Folder... 클릭.
   3. 방금 만든 `C:₩python-workspace` 폴더 선택 → Select Folder.
      > 앞으로 이 폴더가 프로젝트 루트라고 보면 됩니다.

2. 새 파이썬 파일 만들기
   1. VS Code 왼쪽 Explorer(파일 탐색기)에서
      상단의 New File 아이콘(+) 클릭.
   2. 파일 이름을 `hello.py`로 입력 후 Enter.

3. 간단한 코드 작성

```
print("Hello, I wrote Python code in VS Code!")
```

저장(`Ctrl+S`).

5. VS Code에서 파이썬 코드 실행하는 3가지 방법

방법 1) 에디터 상단의 실행 버튼 사용

1. `hello.py`를 열어둔 상태에서
2. 코드 위쪽에 Run Python File 같은 ▶ 버튼이 보이면 클릭.
   VS Code 하단에 **터미널(Terminal)**이 열리면서:
   ...> python c:₩python-workspace₩hello.py
   같은 명령이 실행되고, 출력:
   Hello, I wrote Python code in VS Code!
   이 나오면 성공.

방법 2) VS Code 통합 터미널에서 직접 실행

프롬프트 위치를 알려주기 좋은 방식입니다.

1. 상단 메뉴: View → Terminal (단축키: `Ctrl+``)
2. 아래에 열리는 터미널에서 현재 경로 확인:
   ...> pwd 또는 cd
3. 프로젝트 폴더로 이동 (만약 다른 곳에 있다면):
   ...> cd C:₩python-workspace
4. 파이썬 파일 실행:
   ...> python hello.py
   Hello, I wrote Python code in VS Code!

방법 3) 디버깅 실행 (브레이크포인트 사용)

1. `hello.py`에서 코드 줄 번호 왼쪽 빈 공간을 클릭해서 빨간 점(브레이크포인트) 찍기.
2. 왼쪽 사이드바에서 Run and Debug(▶ bug 모양) 클릭.
3. 상단의 Run and Debug 버튼 클릭.
4. 처음 실행하면 "Python File" 선택.
5. 디버거가 브레이크포인트에서 멈추면:
   * 상단에 재생/일시정지/한 줄씩 실행(▷, ⬇) 버튼이 보임.
   * 왼쪽에 VARIABLES에서 변수 값 확인 가능.

6. Python 인터프리터(버전) 선택하기

VS Code는 어떤 Python을 사용할지 선택해야 할 때가 많습니다.
(특히 여러 버전 설치되어 있을 때)

1. VS Code 아래 상태바 왼쪽에 "Python 3.xx" 처럼 보이는 부분 클릭.
2. 오른쪽 상단에 인터프리터 목록이 뜨면:
   * `Python x.xx (C:₩Pythonxxx₩python.exe)` 처럼 나오는 항목 선택.
3. 만약 아무 것도 안 보이면:
   * "Enter interpreter path" 선택 후
     직접 `python.exe` 경로를 지정할 수도 있습니다.

7. 실제 프로젝트 구조로 만들어 보기

1. 폴더 구조 예시

`C:₩python-workspace` 안에 아래처럼 만들기:

```
python-workspace/
  └── my_project/
      ├── app.py
      ├── utils.py
      └── data/
          └── sample.txt
```

VS Code에서:

1. Explorer에서 `python-workspace` 오른쪽 클릭 → New Folder → `my_project`
2. `my_project` 안에:
   * New File → `app.py`
   * New File → `utils.py`
   * New Folder → `data`
   * `data` 안에 `sample.txt` 생성

2. `utils.py` 작성

```
def greet(name: str) -> str:
    return f"Hello, {name}! Welcome to VS Code."
```

3. `app.py` 작성

```
from utils import greet

def main():
    name = input("이름을 입력하세요: ")
    message = greet(name)
    print(message)

if __name__ == "__main__":
    main()
```

4. 실행

1. 터미널에서:
   ...> cd C:₩python-workspace₩my_project
   ...> python app.py

2. 또는 `app.py` 파일 열고 **Run Python File** 버튼 클릭.

8. 가상환경(venv)까지 사용하는 정석 프로젝트 방식

프로젝트마다 파이썬 패키지를 따로 관리하는 경우에 사용하는 방식입니다.

1. 프로젝트 폴더 만들기

```
...> cd C:₩python-workspace
...> mkdir venv_project
...> cd venv_project
```

2. 가상환경 생성

```
...> python -m venv venv
```

생성 후 폴더 구조:

```
venv_project/
  ├── venv/          # 가상환경 폴더
  └── main.py        # (앞으로 만들 파일)
```

3. 가상환경 활성화 (Windows PowerShell 기준)

```
...> .₩venv₩Scripts₩Activate.ps1
```

프롬프트가 이렇게 바뀝니다:

```
(venv) PS C:₩python-workspace₩venv_project>
```

cmd 터미널:

```
...> .₩venv₩Scripts₩activate.bat
...> mac/Linux: source venv/bin/activate
```

4. VS Code에서 이 프로젝트 열기

1. VS Code에서 File → Open Folder... → venv_project 선택.
2. VS Code가 자동으로 `venv`를 감지하는 경우가 많음.
3. 아래 상태바의 Python 인터프리터를 눌러,
   `venv`가 포함된 Python을 선택:
   예) `Python 3.11인 경우
   (C:₩python-workspace₩venv_project₩venv₩Scripts₩python.exe)

5. 패키지 설치 예시

```
(venv) ...> pip install requests
```

6. `main.py` 작성

```
import requests

def main():
    response = requests.get("https://api.github.com")
    print("상태 코드:", response.status_code)
    print("응답 일부:", response.text[:100])

if __name__ == "__main__":
    main()
```

7. 실행

터미널에서:

...> python main.py

또는 VS Code에서 Run 버튼 사용.

9. 디버그 설정 파일(launch.json) 만들기 (선택사항)

1. 왼쪽 Run and Debug 탭 클릭.
2. "create a launch.json file" 링크 클릭.
3. 환경 선택 화면에서 Python File 선택.

   그러면 `.vscode/launch.json`이 생성되고, 기본 설정이 들어갑니다.

   이제부터 `**F5**`만 눌러도 현재 열려 있는 Python 파일이 디버깅 모드로 실행됩니다.

[초보자가 자주 실수하는 부분]

1. "python 명령이 안 된다"

* PATH 설정이 안 되어 있을 가능성
  → Python 설치할 때 "Add Python to PATH" 체크 안 한 경우.
* 해결: Python 다시 설치("Add to PATH" 체크) 또는 직접 환경변수 수정.

2. VS Code에서 파이썬 확장 설치를 안 해서 실행이 안 되는 경우

* 왼쪽 Extensions에서 Python(Microsoft) 꼭 설치.

3. 프로젝트 루트 경로

* 터미널 프롬프트가 항상 `C:₩python-workspace₩...` 안에 있는지 확인.
* `cd` 명령으로 들어가 있는 위치를 확인하는 습관 중요.

4. 가상환경 활성화를 잊어버린 경우

* 패키지를 설치했는데 "ModuleNotFoundError"가 나면
* 현재 터미널이 venv 활성 상태인지 `(venv)` 표시 확인.

4. Anaconda 설치 및 가상환경 만들어 사용하기

1. 아나콘다 설치

1. 아나콘다 다운로드

- Anaconda 공식 웹사이트 (https://www.anaconda.com/)에 접속
  운영체제에 맞는 설치 파일을 다운로드합니다.

2. 아나콘다 설치

- 설치 프로그램을 실행하고, 안내에 따라 설치를 진행합니다.
- 설치 시 `PATH` 환경변수 설정 여부를 묻는 옵션이 있는데,
  이를 체크하는 것을 권장됩니다.
- 설치가 완료되면 Anaconda Navigator 또는 Anaconda Prompt를 실행합니다.

2. 가상환경 생성

1. Anaconda Prompt 실행

- 시작 메뉴에서 `Anaconda Prompt`를 실행합니다.

2. 가상환경 생성 명령어

--name == -n

--version == -v

conda create --name <가상환경이름> python=<파이썬버전>

- 예: conda create --name myenv python=3.13
- `<가상환경이름>`: 생성할 가상환경의 이름입니다.
- `<파이썬버전>`: 원하는 파이썬 버전(예: 3.13)을 입력합니다.

3. 가상환경 활성화

conda activate <가상환경이름>

- 예: `conda activate myenv`

4. 가상환경 비활성화

conda deactivate

5. 아나콘다에 존재하는 가상환경들 정보 확인

conda info --envs

3. 패키지 설치

1. 필요한 패키지 설치

- 가상환경이 활성화된 상태에서 필요한 패키지를 설치합니다.
  conda install <패키지이름>
- 예: `conda install numpy pandas`

- 특정 버전의 패키지를 설치하려면:
  conda install <패키지이름>=<버전>
  예: `conda install numpy=1.21.0`

2. pip으로 패키지 설치

- conda에서 제공하지 않는 패키지를 설치할 때는 `pip`을 사용합니다.
  pip install <패키지이름>

3. 현재 가상환경에 설치된 패키지 목록 확인하기

(가상환경이름) 경로> conda list

4. 가상환경 안에 설치된 패키지 리스트를 파일로 저장

pip freeze -r 파일명.txt
예 : pip freeze -r requirements.txt
pip freeze -r tensorenv.txt

저장시 경로 지정 가능 예 :
...> mkdir c:₩packagelist
...> pip freeze > c:₩packagelist₩requirements.txt

기본 생성된 파일의 위치 확인 :
....> cd

5. 패키지리스트 저장 파일을 다른 가상환경에서 똑같은 패키지를 설치하려고 할 때 이용할 수 있음

conda install --file [파일명]
예 : (faceloginenv) path...> conda install --file tensorenv.txt

콘다 가상환경 삭제 : (base) 상태에서
conda remove --name <가상환경이름> --all
--all : 가상환경내 모든 패키지를 함께 제거

4. vscode 에서 파이썬 프로젝트에 conda 가상환경 연결하기

터미널:
...> conda --version 확인

방법 1 :
프로젝트 폴더 열기 > ctrl + shift + p : conda 가상환경 선택 지정
> cmd 터미널 열기함 : (conda가상환경) 프롬프트> 표시 확인
방법 2 :
vscode 터미널 > conda create -n 가상환경이름 python=버전
> 직접 만들어서 활성화함

5. Jupyter Notebook

1. Jupyter Notebook 실행

Jupyter Notebook(주피터 노트북)은 Python 및 데이터 분석/AI 실습에서 가장 많이 사용하는 인터랙티브 개발 환경입니다. 실행 방법은 로컬, 가상환경, 클라우드 등에 따라 여러 가지가 있습니다.

1. 기본 실행

- 설치

  pip install notebook
- 실행

  jupyter notebook

  브라우저 자동 실행됨

  기본 주소:

  http://localhost:8888

2. JupyterLab 실행

Notebook 의 업그레이드 버전 (IDE 느낌)

- 설치

  pip install jupyterlab
- 실행

  jupyter lab
- 특징
  - 파일 탐색기 포함
  - 멀티 탭 지원
  - VSCode 스타일 UI

3. Anaconda / Miniconda Prompt 환경에서 실행

- Anaconda 설치 후

  conda install notebook

  jupyter notebook

  또는

  conda install jupyterlab

  jupyter lab

- Navigator GUI 실행
  - Anaconda Navigator 실행
  - "Launch" 버튼 클릭

- **Anaconda 설치 시 포함되는 구성**

  Anaconda 는 다음을 기본 포함합니다.
  - Python
  - Jupyter Notebook
  - JupyterLab
  - 주요 데이터 분석 라이브러리 (NumPy, Pandas 등)

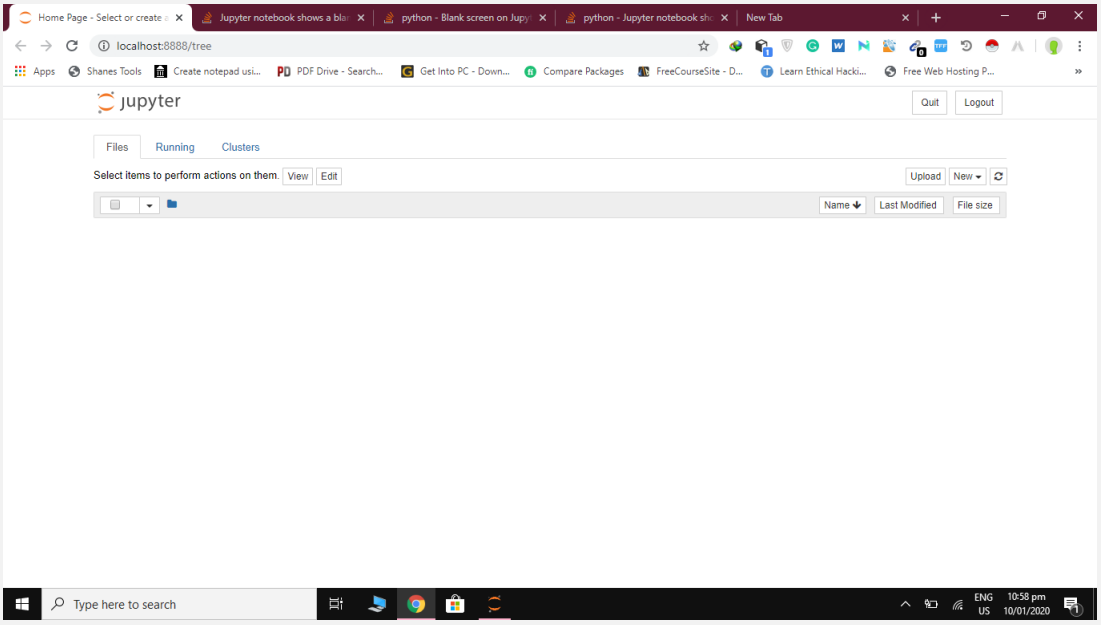
  따라서 pip install notebook 같은 추가 작업 없이 실행할 수 있음

- **Windows 시작 메뉴에서 실행 방법 (가장 쉬운 방법)**

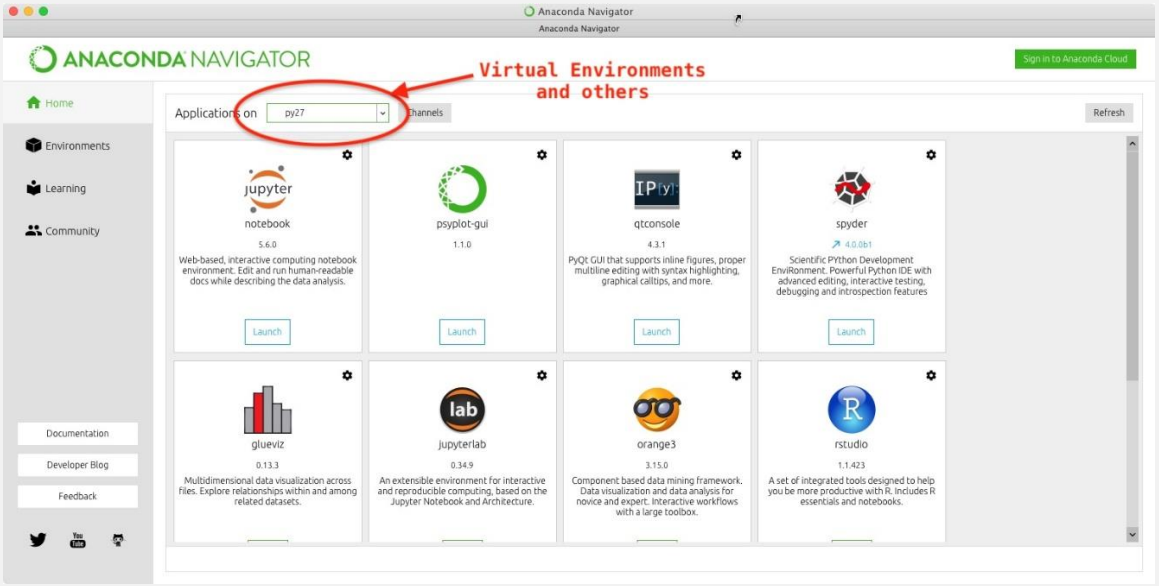
  ✔ **실행 절차**

  1. Windows 시작 버튼 클릭
  2. **Anaconda 폴더 찾기**
  3. 아래 메뉴 중 선택
     - **Jupyter Notebook**
     - **JupyterLab**
     - **Anaconda Navigator**

Jupyter Notebook 실행 화면



Anaconda Navigator 로 실행 (GUI 방식) 실행 화면



4. 가상환경(venv)에서 실행

- 가상환경 생성

  python -m venv venv

- 활성화

  Windows:

  ...> venv₩Scripts₩activate

  Linux/Mac:

  ...> source venv/bin/activate

- 설치 및 실행

  pip install notebook

  jupyter notebook

5. VS Code 에서 실행

- **준비**
  - Visual Studio Code 설치
  - Python Extension 설치
- **실행 방법**
  1. .ipynb 파일 생성
  2. 상단 "Run" 버튼 클릭

- **특징**
  - 코드 + 노트북 통합
  - Git 연동 쉬움
  - 학생 교육에 매우 적합

6. Google Colab (클라우드 실행)

- **사용**
  - Google Colab 접속
    https://colab.research.google.com
- **특징**
  - 설치 필요 없음
  - GPU 무료 사용 가능
  - 협업 가능

Jupyter Notebook 파일 확장자

.ipynb (Interactive Python Notebook)

- **의미**
  - **ipynb = Interactive Python Notebook**
  - JSON 형식으로 저장되는 파일
  - 코드 + 실행 결과 + Markdown(문서) 포함
- **내부 구조**

  .ipynb 파일은 실제로는 **JSON 구조**입니다.

```
{
  "cells": [
    {
      "cell_type": "code",
      "source": ["print('Hello')"]
    }
  ]
```

언제 주피터 노트북 사용이 적합한가

- 알고리즘 실험
- 모델 성능 비교
- 데이터 분석
- 시각화
- 수업 실습
- 코드 예제 설명
- 초기 프로토타입 개발

즉, **아이디어 검증용**으로는 아주 좋습니다.

가장 좋은 실무 방식

1 단계

주피터 노트북에서 아이디어 실험

예를 들어,

- 1 셀: 설정값
- 2 셀: 데이터 로딩
- 3 셀: 전처리 함수
- 4 셀: 모델 클래스
- 5 셀: 실행 코드

2 단계

검증된 코드를 .py 파일로 분리

3 단계

애플리케이션 구조로 재구성

예:

```
project/
    ├── notebooks/
    |       └── experiment.ipynb
    ├── src/
    |       ├── preprocessing.py
    |       ├── model.py
    |       └── app.py
    └── tests/
```

6. Git 에 개인 프로젝트 관리하기

1. 준비 작업

- **Git 설치 (https://git-scm.com/)**
- **GitHub 계정 (https://github.com/, 먼저 회원가입 할 것)**

확인 방법은 VS Code 터미널에서 아래 명령을 입력하면 됩니다.
...> python --version
...> git --version
정상이라면 버전이 표시됩니다.
예:
Python 3.11.9
git version 2.47.0
python 명령이 안 되면 Python 설치 또는 PATH 설정 문제일 수 있습니다.
git 명령이 안 되면 Git 이 설치되지 않았거나 PATH 가 잡히지 않은 상태입니다.

2. VS Code 와 Python 연동 준비

2-1. VS Code 실행

VS Code 를 실행합니다.

2-2. Python 확장 설치 확인

왼쪽 메뉴의 **Extensions(확장)** 를 클릭한 뒤 Python 을 검색해서 **Microsoft 의 Python 확장**을 설치 또는 확인합니다.

- Python
- Pylance
- Jupyter

2-3. Git 이 VS Code 에서 보이는지 확인

VS Code 왼쪽 사이드바에 **Source Control(소스 제어)** 아이콘이 보이면 Git 기능을 사용할 준비가 된 것입니다.

3. 개인 프로젝트 폴더 만들기

예를 들어 아래 폴더를 만든다고 가정하겠습니다.
C:₩python_workspace₩python-study

이 폴더가 **하나의 Git 프로젝트 루트**가 됩니다.
프로젝트 하나당 폴더 하나 원칙으로 가는 것이 좋습니다.
좋은 예:
C:₩python_workspace₩python-study
좋지 않은 예:
C:₩python_workspace
상위 폴더에서 Git 을 시작하면 다른 프로젝트까지 함께 묶일 수 있습니다.

4. VS Code 에서 프로젝트 폴더 열기

VS Code 에서 다음 순서로 진행합니다.

- **File**
- **Open Folder**
- C:₩python_workspace₩python-study 선택

이제 VS Code 가 이 폴더를 현재 프로젝트로 인식합니다.

5. Python 파일 만들기

프로젝트 폴더 안에 예제로 main.py 파일을 하나 만듭니다.
예:

```
print("Hello, Python + Git + VS Code")
```

추가로 README.md 파일도 하나 만들면 좋습니다.
예:

```
# python-study
VS Code, Python, Git 연동 실습용 개인 프로젝트
```

6. 가상환경 만들기

Python 프로젝트에서는 가상환경을 쓰는 것이 안전합니다.
VS Code 터미널을 엽니다.

- 상단 메뉴 **Terminal**
- **New Terminal**

그 다음 프로젝트 루트에서 아래 명령 실행:

```
...> python -m venv .venv
```

이 명령은 .venv 폴더를 만듭니다.

6-1. 가상환경 활성화

Windows PowerShell 기준:

...> .venv₩Scripts₩Activate.ps1

명령 프롬프트(cmd) 기준:

...> .venv₩Scripts₩activate

정상 활성화되면 터미널 앞부분에 보통 이런 표시가 붙습니다.

(.venv)

6-2. VS Code 인터프리터 선택

가상환경을 만든 뒤, VS Code 에서 Python 인터프리터를 선택해야 합니다.

방법:

- Ctrl + Shift + P
- Python: Select Interpreter
- .venv 경로 선택

이 과정을 해야 VS Code 가 이 프로젝트의 Python 환경을 올바르게 사용합니다.

7. .gitignore 파일 먼저 만들기

이 단계가 매우 중요합니다.

가상환경, 캐시, 불필요한 파일이 GitHub 에 올라가지 않도록 막아야 합니다.

프로젝트 루트에 .gitignore 파일을 만듭니다.

Python 프로젝트용 예시는 아래처럼 작성하면 됩니다.

```
# Python cache
__pycache__/
*.py[cod]

# Virtual environment
.venv/
venv/
env/

# VS Code
.vscode/

# Jupyter
.ipynb_checkpoints/
```

```
# OS files
.DS_Store
Thumbs.db

# Environment files
.env
```

중요한 점은 **.venv/를 꼭 넣는 것**입니다.
가상환경 전체를 GitHub 에 올리면 저장소가 커지고 불필요한 파일이 매우 많아집니다.

8. Git 초기화하기

이제 프로젝트 폴더를 Git 저장소로 만듭니다.
VS Code 터미널에서 아래 명령 실행:
...> git init
정상 실행되면 현재 폴더에 .git 이라는 숨김 폴더가 생성됩니다.
이제부터 이 폴더는 Git 이 버전 관리합니다.

9. Git 사용자 정보 설정

처음 한 번은 이름과 이메일을 등록해야 합니다.
...> git config --global user.name "본인이름"
...> git config --global user.email "본인이메일"
예:
...> git config --global user.name "Gil-dong Hong"
...> git config --global user.email "example@gmail.com"
확인:
...> git config --global --list
...> 이름과 이메일이 보이면 정상입니다.

10. 기본 브랜치를 main 으로 맞추기

처음부터 main 브랜치를 쓰는 것이 가장 무난합니다.
...> git branch -M main

11. VS Code 에서 Git 상태 확인하기

이제 왼쪽 **Source Control** 아이콘을 클릭합니다.
그러면 변경된 파일들이 보입니다.
예:

- main.py
- README.md
- .gitignore

즉, VS Code 가 Git 과 연동되어 파일 변경을 추적하기 시작한 것입니다.
터미널에서도 확인 가능합니다.
...> git status

12. 첫 번째 버전 저장하기

Git 에서는 보통 아래 순서입니다.

1. add
2. commit

12-1. add 하기

터미널에서 전체 파일 추가:
...> git add .
또는 VS Code Source Control 화면에서 각 파일 옆의 + 버튼을 눌러 추가할 수도 있습니다.

12-2. 상태 확인

...> git status
이제 Changes to be committed 상태로 보여야 정상입니다.

12-3. commit 하기

...> git commit -m "Initial commit"
이제 첫 버전이 로컬 PC 에 저장되었습니다.

13. GitHub 저장소 만들기

이제 로컬 프로젝트를 GitHub 에 올릴 저장소를 만듭니다.
GitHub 에 로그인한 뒤 새 저장소를 만듭니다.
예:

- Repository name: python-study
- Public 또는 Private 선택

초보자에게는 처음에 **빈 저장소**를 만드는 것이 가장 안전합니다.
즉, 저장소 생성 시 다음은 체크하지 않는 것이 좋습니다.

- Add a README file
- Add .gitignore
- Choose a license

왜냐하면 로컬에 이미 파일이 있는데 GitHub 쪽도 먼저 파일이 생기면 충돌을 겪기 쉽기 때문입니다.

14. 로컬 프로젝트와 GitHub 연결하기

GitHub 저장소를 만들면 주소가 보입니다.
예:
https://github.com/사용자명/python-study.git
이 주소를 복사한 뒤 VS Code 터미널에서 입력합니다.
...> git remote add origin https://github.com/사용자명/python-study.git
예:
...> git remote add origin https://github.com/chajuyun/python-study.git
확인:
...> git remote -v
정상이라면 origin 주소가 표시됩니다.

15. GitHub 에 첫 업로드(push)

이제 GitHub 에 올립니다.
...> git push -u origin main
의미는 다음과 같습니다.

- push: 로컬 커밋을 GitHub 에 업로드
- -u: 이후부터 기본 업로드 대상 기억

처음에는 인증이 필요할 수 있습니다.
브라우저 로그인 창이 뜨면 GitHub 계정으로 로그인하면 됩니다.
정상 완료되면 GitHub 저장소 새로고침 시 파일이 올라가 있습니다.

16. VS Code 에서 이후 버전 관리하는 방법

이제부터는 프로젝트를 수정할 때마다 같은 패턴으로 관리하면 됩니다.

예를 들어 main.py 를 수정했다고 가정합니다.

```
def greet(name):
    print(f"Hello, {name}")

greet("Python")
```

그다음 버전 관리 순서는 아래와 같습니다.

16-1. 변경 확인

VS Code 왼쪽 **Source Control** 을 클릭하면 바뀐 파일이 보입니다.
터미널로도 확인 가능:
...> git status

16-2. 변경 파일 add

전체 추가:
...> git add .
또는 VS Code 에서 파일 옆 + 클릭

16-3. commit 메시지 작성

터미널:
...> git commit -m "Add greet function"
또는 Source Control 상단 메시지 입력창에 적고 Commit 버튼 클릭

16-4. push

...> git push
이제 GitHub 에 새 버전이 반영됩니다.

17. VS Code 만으로 하는 방법

VS Code 에서는 Git 명령을 직접 다 입력하지 않아도 됩니다.

Source Control 사용 흐름

1. 파일 수정
2. 왼쪽 Source Control 클릭
3. 변경 파일 확인
4. + 버튼으로 스테이징

5. 위 입력창에 메시지 작성
6. Commit
7. Sync Changes 또는 Push

하지만 처음에는 **터미널 명령과 VS Code UI 를 같이 익히는 것**이 좋습니다.
그래야 오류가 났을 때 원인을 이해하기 쉽습니다.

18. 추천하는 개인 프로젝트 폴더 구조

Python 개인 프로젝트는 처음에는 아래 구조가 가장 무난합니다.

```
python-study/
  ├── .venv/
  ├── .gitignore
  ├── README.md
  ├── main.py
  └── requirements.txt
```

패키지를 설치했다면 requirements.txt 도 관리하는 것이 좋습니다.
예:
...> pip freeze > requirements.txt
이 파일을 Git 에 올리면 다른 PC 에서도 같은 패키지 환경을 재현하기 쉽습니다.

19. 패키지 설치 후 버전 관리 방법

예를 들어 requests 패키지를 설치했다면:
...> pip install requests
...> pip freeze > requirements.txt
그다음 Git 버전 관리:
...> git add .
...> git commit -m "Add requests dependency"
...> git push
즉, 라이브러리 자체를 올리는 것이 아니라 **requirements.txt** 를 올리는 것입니다.

20. 자주 쓰는 필수 명령어

현재 상태 확인
.git status
변경 파일 추가
git add .

커밋

git commit -m "메시지"

GitHub 업로드

git push

커밋 기록 보기

git log --oneline

원격 저장소 확인

git remote -v

21. 초보자가 가장 자주 하는 실수와 예방법

실수 1. .venv 를 GitHub 에 올림

이유:

- .gitignore 를 안 만들었거나 늦게 만듦

예방:

- Git 시작 전에 .gitignore 먼저 만들기

실수 2. 상위 폴더에서 git init

이유:

- 프로젝트 폴더가 아닌 큰 폴더에서 시작함

예방:

- 반드시 프로젝트 루트 폴더에서만 git init

실수 3. add 없이 commit 하려 함

정확한 순서:

- add
- commit
- push

실수 4. GitHub 저장소에 README 를 먼저 생성함

초보자는 로컬과 원격의 초기 이력이 달라져서 헷갈릴 수 있습니다.

예방:

- 처음엔 GitHub 빈 저장소 생성

실수 5. 비밀번호 파일 업로드

예:

- .env
- API 키
- DB 비밀번호

예방:

- .gitignore 에 .env 추가
- 민감정보는 올리지 않기

22. 이미 .venv 가 추적된 경우 해결법

만약 .gitignore 를 늦게 작성해서 .venv 가 이미 추적되기 시작했다면 아래처럼 처리합니다.

```
git rm -r --cached .venv
git add .
git commit -m "Remove virtual environment from tracking"
git push
```

이렇게 하면 Git 추적 대상에서만 제거되고, 내 PC 의 .venv 폴더는 그대로 남습니다.

23. VS Code 에서 Python 실행과 Git 관리 연결 흐름

실제 개인 프로젝트는 보통 이렇게 진행됩니다.

처음 1 회

1. 프로젝트 폴더 생성
2. VS Code 로 폴더 열기
3. Python 파일 생성
4. 가상환경 생성
5. 인터프리터 선택
6. .gitignore 생성
7. git init
8. git add .
9. git commit -m "Initial commit"
10. GitHub 저장소 생성
11. git remote add origin ...
12. git push -u origin main

이후 반복

1. 코드 수정
2. 실행 테스트
3. git status
4. git add .

5. git commit -m "작업 내용"
6. git push

24. 가장 권장하는 커밋 메시지 방식

초보자는 커밋 메시지를 짧고 명확하게 쓰는 습관이 중요합니다.
좋은 예:
git commit -m "Initial commit"
git commit -m "Add login function"
git commit -m "Fix input error handling"
git commit -m "Update README"
git commit -m "Add requests dependency"
좋지 않은 예:
git commit -m "update"
git commit -m "change"
git commit -m "many fixes"

25. 다른 컴퓨터에서 이어서 작업하는 방법

이미 GitHub 에 올린 프로젝트는 다른 PC 에서도 이어서 작업할 수 있습니다.
git clone https://github.com/사용자명/python-study.git
그다음:

1. VS Code 로 폴더 열기
2. 가상환경 만들기
3. 인터프리터 선택
4. 패키지 설치

예:
python -m venv .venv
.venv₩Scripts₩activate
pip install -r requirements.txt
그 후 평소처럼 수정 → commit → push 하면 됩니다.

26. 초보자용 최종 정리

처음 한 번만 하는 작업

python -m venv .venv
git init

```
git branch -M main
git config --global user.name "본인이름"
git config --global user.email "본인이메일"
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/사용자명/저장소명.git
git push -u origin main
```

이후 반복 작업

```
git status
git add .
git commit -m "작업 내용 설명"
git push
```

27. 가장 안전한 초보자 방식

개인 프로젝트를 VS Code + Python + Git 으로 관리할 때는 아래 원칙을 지키면 거의 꼬이지 않습니다.

- **프로젝트 하나당 폴더 하나**
- **가상환경은 .venv**
- **.gitignore 먼저 작성**
- **VS Code 에서 인터프리터 선택**
- **커밋은 기능 단위**
- **push 자주 하기**
- **비밀정보는 올리지 않기**